

Web Systems Development

What a Framework Gets you: Laravel

RPI

ITWS 2110 · QUIZ 1 REVIEW

Seven weeks on a few slides

The web stack, Docker, testing, CSS and Tailwind, React, and the ideas behind a framework. One slide per topic: what to be able to explain, and what to be able to trace.

Quiz 1 is Tuesday 10/13, in class.

How the quiz works

Format

- On paper, offline. No laptops, phones or other devices
- Multiple choice: one best answer, A to E
- Answers go on a Scantron sheet. Bring a pencil

Your crib sheet

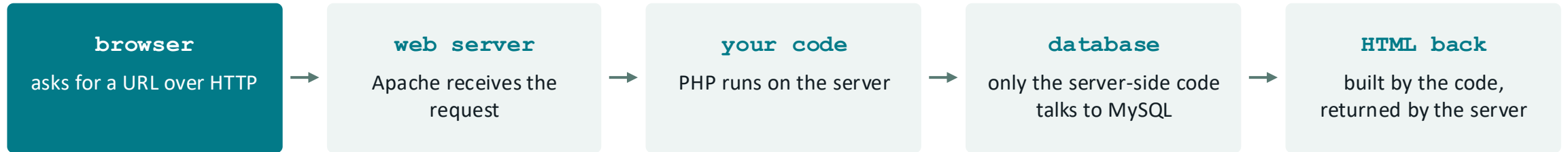
- One 8.5 × 11 page, both sides if you like
- Your own notes. Nothing else on the desk
- Making it is most of the studying

What the questions ask

- Here is some code or a setup. What does it do?
- Here is a symptom. What is the cause?

topic	sessions	from
The web stack	1	how a request is answered
Docker and Compose	2–3	Homework 1
Testing at three levels	4–5	the Pantry app, Homework 2
CSS and Tailwind	6–7	Homework 3
React: components, props, state, state structure, component tests	8–11	the workshops, Homeworks 4 and 5
What a framework buys you	12	Tuesday's lecture: the ideas, not Laravel syntax

Who answers a request, and how that changed



Serve a file

- The first web servers returned the file the URL named
- Same answer for everyone

Run a program

- CGI: a request can run a program
- Now the answer can depend on the request: search results, a login

Be the program

- A framework: the repeated plumbing of web apps, written once, with opinions

Ship the image

- A container: the image you tested is the image that ships
- Ends drift between laptop and production

The browser never talks to the database. It speaks HTTP to the server, and everything else happens behind it.

Images, containers, and what survives

What a container is

- Isolated processes on the host's kernel
- Not a virtual machine: no kernel of its own
- Three containers on one Linux host: one kernel

Build time or start time

- FROM, COPY, RUN happen once, when the image is built
- CMD is the process each container starts with
- COPY is a snapshot: later edits need a rebuild, a bind mount or Compose Watch

What survives

- docker restart: the container's files are still there
- docker rm: the container's filesystem is gone
- Data that must outlive a container goes in a volume

mount	what it is	use it for
bind mount	a folder on your laptop, visible inside the container	source code: edit and see it
named volume	storage Docker manages, kept until you remove it	database files

Typical question: build once, start three containers. How many times does RUN run? How many times does CMD?

Several containers that find each other

docker-compose.yml, THE SHAPE OF HOMEWORK 1

```
services:
  web:
    build: .
    ports: ["8080:80"]      # laptop:container
    depends_on:
      db: { condition: service_healthy }
  db:
    image: mysql:8.4
    volumes: [dbdata:/var/lib/mysql]
volumes: { dbdata: {} }
```

- Services reach each other by service name. From web the database is db, not localhost. Inside a container, localhost is that container.
- 8080:80 is laptop port, then container port. Your browser uses localhost:8080. Another container uses web:80.
- A service with no ports can still be reached by other services. Publishing is only for the laptop.
- Plain depends_on waits for the container to start. service_healthy waits for the database to be ready.
- Init scripts run only when the data folder is empty. To start fresh: docker compose down -v, which also deletes the data.
- Commit .env.example. Keep the real .env out of git.

“My schema change isn't showing up” is almost always the old volume.

Typical question: the page can't reach the database although both containers are running. Which line is wrong?

Three levels, three different questions

level	the question it asks	tool in the Pantry app	what only it can catch
Unit	Does this rule work?	PHPUnit, calling a class directly	a wrong calculation inside one class
API	Is the rule exposed correctly over HTTP?	Playwright's request client, no browser	a wrong status code or response body
Browser (end to end)	Does it work for a person using the page?	Playwright driving Chromium	a broken page: a renamed class, a missing button

The pyramid

- Many fast unit tests
- Fewer slow browser tests, kept for whole journeys
- Check a rule at the lowest level that can see it

What green means

- Every claim the tests make is true right now
- It does not mean every claim was written
- A test with no assertion claims nothing

Red first

- A new test for a missing rule should fail before you fix the code
- A test that was never red may not be testing anything

Typical question: this one thing changed. Which levels go red?

Reading a test, and reading its failure

ARRANGE, ACT, ASSERT

```
public function test_buying_adds_to_the_amount(): void
{
    $stock = new Stock(2);           // arrange
    $stock->add(3);                   // act
    $this->assertSame(5.0, $stock->amount()); // assert
}
```

THE MESSAGE: WHAT IT GOT, THEN WHAT IT EXPECTED

```
Failed asserting that -1.0 is identical to 5.0.
```

- `assertSame` compares value and type (`===`). 5 is not 5.0. `assertEquals` compares with `==`.
- A test stops at its first failed assertion. Later ones never run.
- A test name is a sentence that should be true.
- Playwright: a locator is a description of how to find something. It is looked up when an action or an expect uses it, not when it is written.
- `await expect(locator)` keeps checking for a few seconds. `expect(value)` checks once.
- Each test makes its own data, under a name nothing else uses. That is what `Date.now()` in a product name is for.

Trace it: change `+=` to `-=` in `add()`. The test starts at 2, subtracts 3, and reports `-1.0`.

Which rule wins

EVERYTHING ON THE PAGE

```
.nav a      { color: blue; }      /* 0,1,1 */
a.link      { color: red;  }      /* 0,1,1 */
#main a     { color: green; }     /* 1,0,1 */
.nav .link  { color: purple; }    /* 0,2,0 */
```

```
<div id="main">
  <nav class="nav">
    <a class="link" href="/">Home</a>
  </nav>
</div>
```

- Every rule is global: it applies on every page that loads the stylesheet.
- Specificity is counted as ids, classes, types. Compare left to right: one id beats any number of classes.
- On a tie, the rule written later in the stylesheet wins. The order of names inside class="..." means nothing.
- An inline style beats every selector.
- Text properties such as color are inherited by children. Box properties such as border, padding and margin are not.
- In DevTools, a declaration struck through lost to another rule.

Here the link is green. Delete the #main a rule and it is purple: 0,2,0 beats the two rules at 0,1,1.

Utilities, layout, and mobile first

Utilities

- A class says what the element looks like: px-2, rounded-full
- Spacing steps are 0.25rem: gap-4 is 1rem

Flex and grid

- flex: one row. justify-* works along it, items-* across it
- flex-col swaps the axes: justify-* is now vertical
- justify-between: first child at one end, last at the other
- grid grid-cols-3: equal columns, the grid decides the size

Mobile first

- No prefix: every width, starting with phones
- md: means 768px and wider. lg: means 1024px and wider
- A prefix is a min-width query, never a maximum

classes	at 500px	at 800px	at 1100px
grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3	1 column	2 columns	3 columns
hidden md:block	not shown	shown	shown
flex flex-col md:flex-row	stacked	side by side	side by side
grid grid-cols-3 lg:grid-cols-1	3 columns	3 columns	1 column (the reverse of what was meant)

Components, props and state

Components and props

- `UI = f(data)`: describe the screen for the current data
- A component's name starts with a capital. `<product />` is treated as an HTML tag
- Props come in from the parent and are read-only
- `expired="false"` is a string, and a non-empty string is truthy. Write `expired={false}`

State

- State survives a re-render. An ordinary `let` starts over each time
- A setter requests another render. The variable does not change in the render that is running
- `setCount(count + 1)` three times adds 1. `setCount(c => c + 1)` three times adds 3
- Each instance of a component has its own state

Events, inputs, lists

- `onClick={handleClick}` passes the handler. `onClick={handleClick()}` calls it during render
- `value` without `onChange` is a read-only field
- Items from `.map` need a key from your data: `item.id`, not the index
- `{count && <Badge />}` prints 0 when `count` is 0

Trace it: let `clicks = 0` and `useState(0)`, both increased in the handler. After two clicks: `count` is 2, `clicks` shows 0.

Where state lives

the idea	the rule	what goes wrong otherwise
Derived state	If you can compute it from state you have, don't store it	items and total kept separately: Remove forgets total, and the count is wrong
Lifting state	Put it in the lowest component that contains everyone who needs it	two components each keep their own copy and disagree
One status	One value, 'sending' or 'sent', not two booleans	both booleans true at once: a state that should be impossible
Store the id	Keep the selected item's id and look the item up each render	a stored copy goes stale when the item changes
Reset with a key	A different key is a different component, with fresh state	useState(prop) reads the prop only on the first render, so an editor keeps the old text
Rules of hooks	Top level only: the same hooks, in the same order, every render	a hook inside an if: React throws when the condition changes
Custom hooks	They share logic, not state: every call gets its own	two components call useQuantity() and expect one shared number

Homework 5 was the first four rows: Remaining, the filter, the Details panel, and the title editor with its key.

Component tests

RENDER, ACT, ASSERT

```
test('checking a to-do lowers Remaining', async () => {
  const user = userEvent.setup();
  render(<TodoApp />);

  await user.click(
    screen.getByRole('checkbox', { name: 'Buy rice' }));

  expect(screen.getByText('Remaining: 1'))
    .toBeInTheDocument();
});
```

- Find things the way a person would: by role first, then label, then text. Class names last.
- Every action needs its `await`. Without it, the assertion can run before React re-renders and see the old page.
- `getBy...` throws when it finds nothing, and when it finds more than one.
- To assert that something is gone, use `queryBy...`, which returns null.
- `user.type` adds to what is already in the field. Clear it first if you want to replace it.
- Several buttons with the same name? Find the row, then look `within(row)`.

A test that would also pass on the unfinished app tells you nothing about your work.

The ideas behind Laravel

Who calls whom

- A library you call. A framework calls you
- Inversion of control: the framework owns the flow and decides when your code runs
- Dependency injection: what your code needs is handed to it, not built by it

One request

- index.php → middleware → router → controller → view
- The framework owns the first three. You write the last two
- One entry point: every request gets the same setup and security
- PHP starts fresh for every request. Nothing in a variable survives

MVC and conventions

- The model knows, the view shows, the controller coordinates
- A routing table: method + URL → code. No match is a 404
- Convention over configuration: every app is laid out the same way

the attack	what stops it
Cross-site scripting: a visitor's <script> runs in your page	escaping output. {{ }} does it by default, and only for what goes through a template
SQL injection: input becomes part of a query	bound parameters, which keep input separate from the SQL
Cross-site request forgery: another site submits your form	a secret token in the form, checked by middleware

Concepts only. You will not need to remember Laravel syntax.

Making the page worth bringing

Worth the space

- Rules with a one-line reason: later rule wins on a tie
- Small worked examples you can adapt: a specificity count, a port mapping
- The mistakes you made in the homeworks, and what fixed them
- Pairs that are easy to swap: RUN and CMD, getBy and queryBy, assertSame and assertEquals

Not worth the space

- Long definitions copied from slides
- History and dates
- Syntax you can already read when you see it
- Anything you have not understood yet: fix that, don't copy it

How to practise

- Do the practice questions with your draft sheet beside you
- For each one you miss: what would have had to be on the sheet?
- Re-do a workshop drill you found hard, without looking at the answer
- Explain one slide of this deck to someone else

Bring: a pencil, and one 8.5 × 11 page.